PorraGP - Plan del proyecto

Oct 22, 2026 · @a

PorraGP es una porra de Formula 1 con puntos ficticios para la practica de DAW: la hacemos 4 personas, en JavaScript sin frameworks, con Express y PostgreSQL, y la entregamos antes de Navidad.

La idea

PorraGP es una web para apostar puntos ficticios, nunca dinero real, a carreras de Formula 1. El admin se inventa el calendario y los resultados, y los usuarios compiten en una clasificacion por temporada.

Antes de cada carrera, cada usuario apuesta puntos a quien hara la pole, el podio y la vuelta rapida. Si acierta gana puntos, y si falla los pierde.

Como se juega

Se apuesta con los puntos de la clasificacion: cada usuario empieza la temporada con 1.000 y los gana o los pierde apostando.

- **Puntos:** todos empiezan la temporada con 1.000, tambien quien entra con la temporada empezada. No hay saldo aparte.
- **Que carrera se juega:** solo la proxima. Se abre cuando la anterior tiene resultado (o se cancela) y se cierra a la hora que fije el admin.
- **La jugada:** se apuesta a cinco cosas: pole, primero, segundo, tercero y vuelta rapida. Al menos una apuesta, y como mucho una por cada cosa. Cada apuesta es un piloto y unos puntos, minimo 10. En el podio no se puede repetir piloto.
- **Cuanto se apuesta:** no hay tope, se pueden apostar todos los puntos. Con 100 o menos, se pueden apostar 100 igualmente.
- **Cambios:** la jugada se puede cambiar o borrar hasta el cierre.
- **Minimo de 100:** al cerrar, a quien ha jugado y tiene menos de 100 se le suben los puntos a 100, y despues se descuenta lo apostado. Crea puntos a proposito, para que nadie se quede sin poder jugar.
- **Comodin:** uno por temporada. No se puede quitar una vez activado, la jugada que lo lleva no se puede borrar y la web pide confirmacion antes de activarlo. Si la carrera se cancela, se devuelve.

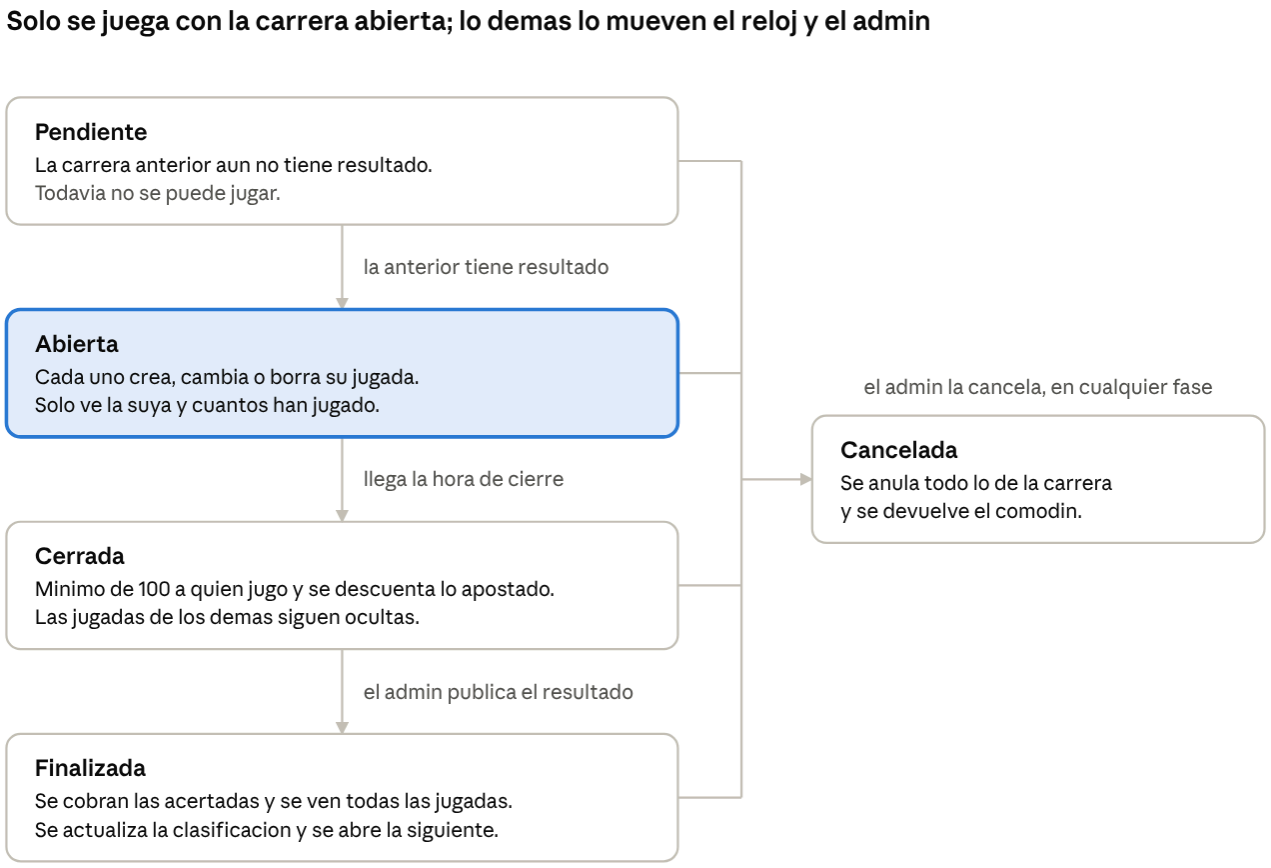
- **Clasificacion:** por puntos. Los empatados comparten puesto (1, 2, 2, 4) y se ordenan por nombre. Quien no ha jugado ninguna carrera sale al final como "sin jugar", sin puesto, y el admin no aparece. Se pueden consultar las temporadas anteriores, y cada grupo privado tiene su propia clasificacion con los mismos puntos.

Lo que cobra cada apuesta:

Caso	Se cobra
Apuesta acertada	x2: se gana lo mismo que se aposto
Podio exacto, con apuesta a primero, segundo y tercero	x3 en cada una de las tres
Apuesta acertada con comodin	x4
Podio exacto con comodin	x6 en cada una de las tres
Apuesta fallada	Nada: se pierde lo apostado

Ejemplo: con 1.000 puntos, Ana apuesta 100 a la pole y 50 al tercero. Al cerrar se le descuentan 150 y se queda con 850. Acierta la pole y falla el tercero: cobra 200 y termina con 1.050.

Una carrera de principio a fin



fases de una carrera · 5 estados

La fase no se guarda en ningun sitio: el servidor la calcula en cada peticion con la hora actual, si la carrera anterior tiene resultado y si esta cancelada.

Usuarios y permisos

Hay tres tipos de usuario, y todos los permisos se comprueban en el servidor: ocultar botones en la web no basta.

Quien	Que puede hacer
Visitante, sin sesion	Ver la portada, registrarse, iniciar sesion y recuperar la contraseña
Usuario	Jugar, ver la clasificacion, los resultados y los perfiles, cambiar su nombre, su piloto favorito y su contraseña, y crear grupos o unirse a ellos
Miembro de un grupo	Ver el grupo, sus miembros y su clasificacion, y salir de el

Quien	Que puede hacer
Creador de un grupo	Ademas, expulsar miembros, regenerar el codigo y borrar el grupo. No puede salir sin borrarlo
Admin	Gestionar temporadas, pilotos, carreras y resultados. No juega, no entra en grupos ni sale en la clasificacion, porque es quien se inventa los resultados

El perfil publico muestra nombre, puntos, posicion y piloto favorito. El historial de jugadas solo lo ve su dueño.

Lo que responde el servidor cuando algo no esta permitido:

- Sin sesion, en cualquier cosa que no sea la portada, el registro, el inicio de sesion o la recuperacion de contraseña: 401.
- Cambiar o borrar la jugada de otro: 403.
- Un usuario haciendo algo de admin, o el admin intentando jugar o entrar en un grupo: 403.
- Pedir la jugada de otro antes del resultado: 404, para no revelar que existe. Antes del resultado, el listado solo trae la tuya; despues, todas.
- Pedir un grupo del que no eres miembro: 404.
- Expulsar, regenerar el codigo o borrar un grupo sin ser su creador: 403.

Modelo de datos

Hay doce tablas. Cada una la lleva una persona, que es quien escribe su SQL.

Tabla	Campos	Reglas	Lleva
USUARIO	id, nombre de usuario, email, contraseña (hash), rol (usuario o admin), piloto favorito, fecha de alta	Nombre y email unicos	Abraham
RECUPERACION	id, usuario, hash del token, caducidad, usado	Un solo uso	Abraham
TEMPORADA	id, año, nombre		Alvaro
PILOTO	id, nombre, nacionalidad	Con datos no se borra	Alvaro

Tabla	Campos	Reglas	Lleva
INSCRIPCION	temporada, piloto, equipo, dorsal, activo	Un piloto una vez por temporada; dorsal unico en la temporada	Alvaro
CARRERA	id, temporada, ronda, nombre del Gran Premio, circuito, hora de cierre, fecha de la carrera, cancelada, cierre procesado	Con jugadas no se borra, se cancela	Alvaro
JUGADA	id, usuario, carrera, comodin, fechas de creacion y de ultima modificacion	Una por usuario y carrera	Marko
APUESTA	id, jugada, categoria, piloto, puntos, estado (pendiente, acertada o fallada)	Una por jugada y categoria; minimo 10 puntos	Marko
GRUPO	id, nombre, codigo de invitacion, creador, fecha	Codigo unico	Marko
MIEMBRO	grupo, usuario, fecha de union	Uno por grupo y usuario	Marko
RESULTADO	carrera, pole, primero, segundo, tercero, vuelta rapida, fecha de publicacion	Uno por carrera; podio sin repetidos	Alex
MOVIMIENTO	id, usuario, temporada, tipo (minimo, apuesta, cobro, anulacion o correccion), puntos (positivos o negativos), carrera, apuesta, fecha	Los puntos de un usuario son la suma de sus movimientos	Alex

Todas las relaciones van con claves foraneas. El servidor comprueba ademas que apuestas y resultados solo usen pilotos inscritos en la temporada de su carrera. Las sesiones van en una tabla aparte que crea connect-pg-simple.

Tecnologias

Todo en JavaScript y sin frameworks en el front, con lo que se ha dado en la asignatura: cualquiera de los 4 puede tocar cualquier parte.

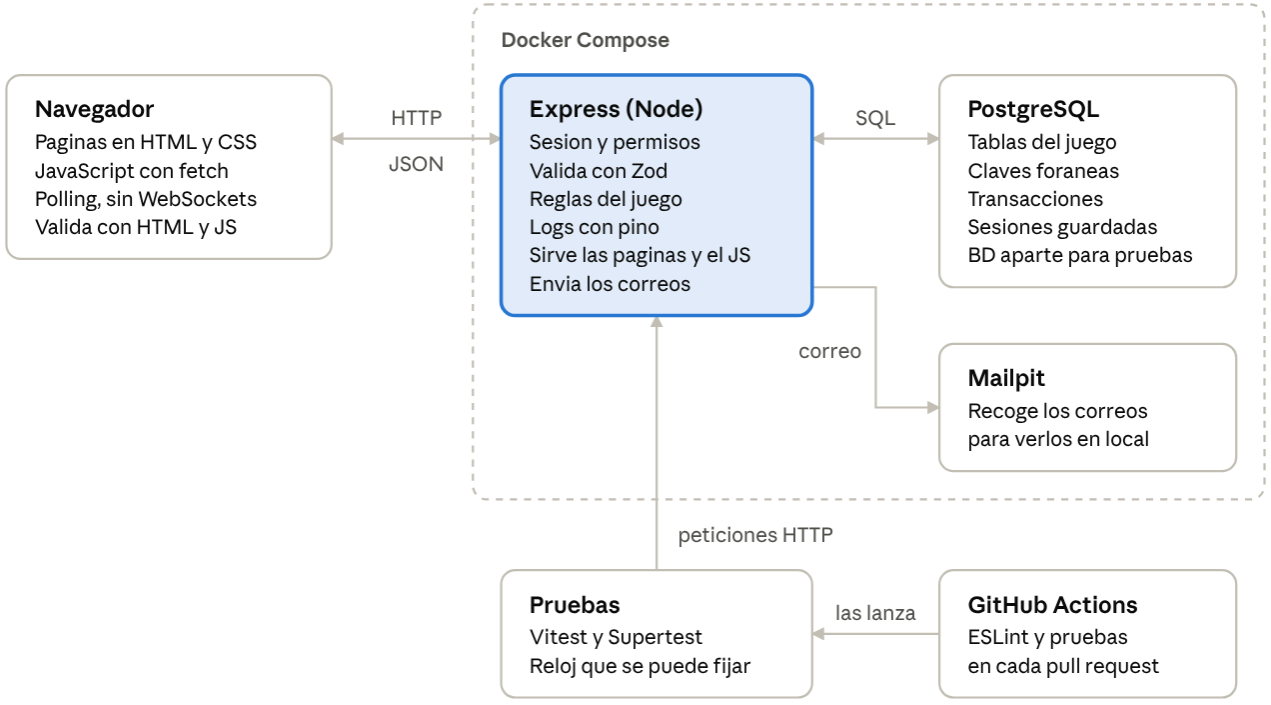
Parte	Tecnologia	Por que
Front	Paginas HTML, CSS y JavaScript sin frameworks, con los modulos nativos del navegador	Es lo que se ha dado en la asignatura: no hay que aprender un framework ni compilar nada
Datos en el front	fetch, con un modulo comun que trata los errores, y setInterval para el polling	El PDF prohibe WebSockets, y con el modulo comun ninguna pagina da por hecho que la peticion sale bien
Estilos	CSS propio y elementos HTML nativos (formularios, tablas, dialog)	Es lo mas facil de hacer accesible y adaptable a movil
Servidor	Node (version LTS) con Express	El mismo lenguaje que el front, y el mas documentado
Base de datos	PostgreSQL con el paquete pg y SQL escrito a mano, sin ORM	La conoceis; los cierres y los cobros van en transacciones y su SQL se explica bien en el oral
Esquema y datos de ejemplo	Ficheros SQL	Se cargan igual en desarrollo y en pruebas
Validacion	En el navegador, atributos HTML (required, min, type) y JavaScript; en el servidor, Zod	El PDF pide validar en los dos lados; las constantes, como la apuesta minima, estan en un modulo compartido para que no se contradigan
Sesiones	express-session, con las sesiones guardadas en PostgreSQL (connect-pg-simple) y cookie HttpOnly	Mas sencillo y seguro que JWT para una web en un solo dominio
Contraseñas	argon2	Argon2id es el algoritmo que recomienda OWASP para guardarlas
Correo	Nodemailer, con Mailpit en local	Mailpit recoge los correos de recuperacion y los enseña en una web, sin enviarlos de verdad
Logs	pino con pino-http (pino-pretty en desarrollo)	En JSON y con un id por peticion

Parte	Tecnologia	Por que
Pruebas	Vitest y Supertest, con una base de datos de pruebas aparte y un reloj que se puede fijar	Prueban el servidor por HTTP, como pide el PDF, y los cierres sin esperar
Calidad	ESLint y Prettier	El mismo estilo de codigo para los 4
Ejecucion	Docker Compose con PostgreSQL, Mailpit y la app	Un comando lo arranca todo; Express sirve tambien las paginas
Repositorio	GitHub con GitHub Actions	Cada pull request pasa ESLint y las pruebas

React daria componentes reutilizables y pantallas que se actualizan solas, pero habria que aprenderlo en 8 semanas. Spring Boot o Django tambien valdrian para el servidor, pero obligarian a llevar dos lenguajes. No usamos TypeScript, para no dar pie a dudas con el "se utiliza JavaScript" del PDF. La aplicacion se ejecuta solo en local.

Arquitectura

El navegador solo habla con Express, y solo Express toca la base de datos



arquitectura · navegador, Express, PostgreSQL y pruebas

Todo se arranca con docker compose up. Las pruebas mandan peticiones HTTP a Express igual que el navegador, pero contra una base de datos aparte. En local, los correos de recuperacion de contraseña llegan a Mailpit.

Logs, pruebas y accesibilidad

Los tres los pide el PDF. Cada persona los cubre en su parte siguiendo estas reglas comunes.

Logs, con pino:

- Cada peticion: metodo, ruta, codigo, duracion, id de peticion e id de usuario si hay sesion.
- Eventos importantes: registro, inicio de sesion correcto y fallido, jugadas, comodin, cierres, resultados, cancelaciones, grupos, recuperaciones de contraseña y cambios del admin.
- Errores no controlados, con su traza.
- Niveles: error para fallos del servidor, warn para peticiones rechazadas (4xx) e inicios de sesion fallidos, info para lo normal y debug solo en desarrollo. El nivel se elige en el .env (LOG_LEVEL).
- Nunca se registran contraseñas, hashes, cookies ni tokens.
- Los logs no sustituyen a los movimientos: los puntos salen siempre de la tabla de movimientos.

Pruebas, con las condiciones del PDF:

- Se lanzan con un unico comando, documentado en el README.
- Usan una base de datos de pruebas aparte, nunca la de desarrollo.
- Cada prueba prepara sus propios datos y no depende del orden.
- Comprueban el codigo, la respuesta y el estado de los datos. Tras un error, comprueban que nada ha cambiado.
- Hay al menos un caso por cada tipo de permiso, y el flujo completo de principio a fin.

Accesibilidad y movil, que el PDF exige, con WCAG 2.2 nivel AA como referencia:

- HTML semantico, idioma de la pagina en español y encabezados ordenados.
- Todo usable con teclado, con el foco visible.
- Cada campo con su etiqueta y su error al lado.
- Los cambios importantes (jugada guardada, cierre, resultado) se avisan tambien a los lectores de pantalla.
- Aciertos y fallos con icono y texto, no solo con color, y contraste suficiente.
- La clasificacion es una tabla en escritorio y se adapta en movil sin scroll horizontal.

- Confirmacion antes de lo que no se puede deshacer: activar el comodin, borrar una jugada, publicar o corregir un resultado y cancelar una carrera.
- Estados de carga, vacio y error siempre a la vista.
- Se revisa con teclado, con axe o Lighthouse, y en escritorio y en movil.

Como trabajamos

Una rama por tarea, una pull request revisada por alguien de otra parte, y nada entra en main sin que pasen ESLint y las pruebas.

- **Repositorio:** GitHub, con main protegida. Carpetas: public (paginas HTML, CSS y JavaScript del navegador), server (Express), shared (constantes comunes, como las categorias, los multiplicadores y la apuesta minima) y db (ficheros SQL).
- **Pull requests:** pequeñas, una por tarea, y revisadas por alguien de otra parte. Asi todos conoceis todo el codigo para el oral.
- **CI:** GitHub Actions pasa ESLint y las pruebas, con PostgreSQL, en cada pull request.
- **Tarea hecha:** funciona, tiene sus pruebas, se usa con teclado y se ve bien en movil.
- **README:** cada uno documenta su parte: sus endpoints y como se prueba.
- **Secretos:** en un .env fuera del repositorio, con un .env.example de plantilla.
- **Datos de usuarios en pantalla:** se pintan con textContent, nunca con innerHTML, para que nadie pueda colar codigo (XSS).
- **Errores de la API:** todos con el mismo formato y estos codigos.

Codigo	Cuando
400	Datos invalidos o incompletos
401	Sin sesion
403	Sin permiso
404	No existe o no se puede ver
409	Choca con lo que ya hay: datos repetidos, carrera no abierta, comodin ya usado, puntos insuficientes o borrar algo con datos relacionados

Reparto

Cada persona lleva una parte entera, con sus pantallas, su API, sus tablas y sus pruebas. Los nombres son provisionales: cambiadlos por los vuestros.

Persona	Parte	Tablas	Pantallas principales
Abraham	Usuarios, recuperacion de contraseña y componentes comunes	USUARIO, RECUPERACION y sesiones	Portada, registro, inicio de sesion, recuperar la contraseña y perfiles
Alvaro	Admin, infraestructura y base del front	TEMPORADA, PILOTO, INSCRIPCION y CARRERA	Estructura comun de las paginas, calendario y pantallas del admin
Marko	Jugadas y grupos privados	JUGADA, APUESTA, GRUPO y MIEMBRO	Proxima carrera, formulario de jugada, historial y grupos
Alex	Puntos y clasificaciones	RESULTADO y MOVIMIENTO	Clasificacion general y de cada grupo, resultado de cada carrera y publicar el resultado

La carga queda parecida entre los cuatro: Marko suma los grupos, Alvaro la base del front y el calendario, y Abraham la recuperacion de contraseña.

Los endpoints de cada parte son una primera propuesta: se cierran entre todos en la semana 1.

Abraham: usuarios y componentes comunes

Abraham hace que la gente pueda registrarse, entrar, recuperar la contraseña y tener su perfil, y los componentes accesibles que usan todas las paginas. Necesita de Alex los puntos y la posicion para el perfil publico.

Lo primero, para los demas (semanas 1 y 2)

- [ ] Comprobaciones de sesion y de admin en el servidor (requireAuth y requireAdmin), que usan todos los endpoints.
- [ ] Componentes comunes accesibles en JavaScript: dialogo de confirmacion, avisos para lectores de pantalla y estados de carga, vacio y error.

Servidor

- [ ] POST /api/auth/registro: nombre y email unicos, contraseña guardada con argon2.
- [ ] POST /api/auth/login y POST /api/auth/logout, con la sesion guardada en PostgreSQL.
- [ ] GET /api/auth/yo: el usuario de la sesion.

- [ ] POST /api/auth/recuperar: envia por correo un enlace con un token aleatorio de un solo uso que caduca y se guarda como hash. Responde lo mismo exista o no el email.
- [ ] POST /api/auth/restablecer: pone la contraseña nueva con ese token y lo invalida.
- [ ] Envio de correos con Nodemailer, y Mailpit en el Docker Compose para verlos en local.
- [ ] GET /api/usuarios/:id: perfil publico (nombre, puntos, posicion y piloto favorito).
- [ ] PATCH /api/usuarios/yo: cambiar el nombre y el piloto favorito.
- [ ] PUT /api/usuarios/yo/contrasena: cambiar la contraseña pidiendo la actual.
- [ ] Logs de registro, inicio de sesion correcto y fallido, recuperacion y cambio de contraseña, sin datos sensibles.

Pantallas

- [ ] Portada publica.
- [ ] Registro e inicio de sesion, con el error junto a cada campo.
- [ ] Recuperar la contraseña y poner una nueva desde el enlace del correo.
- [ ] Perfil publico de un usuario.
- [ ] Mi perfil: editar los datos y cambiar la contraseña.
- [ ] Paginas de error: no encontrado y sin permiso.

Pruebas

- [ ] Sin sesion, las operaciones protegidas dan 401 y no cambia nada.
- [ ] Registro con nombre o email repetido: 409. Con datos invalidos: 400.
- [ ] Inicio de sesion correcto y con contraseña mala.
- [ ] Cambiar la contraseña sin la actual correcta da error y no la cambia.
- [ ] Recuperacion: el token sirve una sola vez y caduca, y la respuesta es la misma exista o no el email. En las pruebas el correo se simula y se lee el enlace de ahi.
- [ ] El perfil publico no enseña datos privados.
- [ ] Nadie puede registrarse como admin.

Alvaro: admin, infraestructura y base del front

Alvaro monta el proyecto para que todos puedan trabajar, la base comun de las paginas y todo lo que gestiona el admin. Necesita de Marko la fase de cada carrera y de Alex la anulacion de una carrera cancelada.

Lo primero, para los demas (semanas 1 y 2)

- [ ] Repositorio en GitHub con public, server, shared y db, y con ESLint y Prettier configurados.
- [ ] Docker Compose con PostgreSQL y la app, y la base de datos de pruebas aparte.
- [ ] Esquema inicial de la BD, juntando el SQL de las tablas de cada uno.
- [ ] GitHub Actions: ESLint y pruebas en cada pull request.
- [ ] Logs con pino y pino-http, y el formato comun de errores con su gestor en Express.
- [ ] Seed con datos inventados: una temporada, pilotos, carreras (las pasadas, con resultado) y un admin de prueba.
- [ ] Base del front: estructura de las paginas HTML, CSS comun, cabecera y menu compartidos, y el modulo de fetch que trata los errores de la API.

Servidor

Leer temporadas, pilotos y carreras puede hacerlo cualquier usuario con sesion; crear, cambiar y borrar, solo el admin.

- [ ] Temporadas: listar, ver, crear, cambiar y borrar en /api/temporadas.
- [ ] Pilotos: lo mismo en /api/pilotos.
- [ ] Inscripciones de pilotos en cada temporada, con equipo, dorsal y activo, en /api/temporadas/:id/inscripciones.
- [ ] Carreras: lo mismo en /api/temporadas/:id/carreras y /api/carreras/:id.
- [ ] La hora de cierre solo se cambia con la carrera pendiente o abierta.
- [ ] Borrar algo con datos relacionados da 409: una carrera con jugadas se cancela, y un piloto con datos se desactiva.
- [ ] POST /api/carreras/:id/cancelar, que llama a la anulacion de Alex.
- [ ] Logs de cada cambio del admin.

Pantallas

- [ ] Calendario de la temporada, con la fase de cada carrera.
- [ ] Admin: temporadas, pilotos e inscripciones, con listas y formularios.
- [ ] Admin: carreras, con lista, crear, cambiar la hora de cierre, borrar y cancelar con confirmacion.

Pruebas

- [ ] CRUD de temporadas, pilotos y carreras, comprobando la base de datos en cada paso.
- [ ] Un usuario normal en cualquier operacion de admin: 403.

- [ ] Datos invalidos o repetidos (campos vacios, dorsal repetido): 400 o 409, y no se guarda nada.
- [ ] Borrar un piloto con apuestas o una carrera con jugadas: 409, y siguen ahi.
- [ ] Cambiar la hora de cierre de una carrera cerrada: 409.

Marko: jugadas y grupos

Marko hace todo lo que pasa mientras una carrera esta abierta, con sus reglas y el comodin, y los grupos privados. Necesita de Alex los puntos disponibles y la clasificacion de cada grupo, y de Alvaro las carreras y los pilotos.

Lo primero, para los demas (semana 2)

- [ ] El reloj del servidor: una unica funcion para la hora actual, que en las pruebas se puede fijar.
- [ ] La fase de cada carrera (pendiente, abierta, cerrada, finalizada o cancelada) y cual es la proxima. La usan Alvaro y Alex.

Servidor: jugadas

- [ ] GET /api/temporadas/:id/proxima: la carrera abierta, su hora de cierre y cuantos han jugado.
- [ ] POST /api/carreras/:id/jugadas: crear la jugada con sus apuestas.
- [ ] PUT y DELETE /api/jugadas/:id: cambiarla o borrarla hasta el cierre.
- [ ] Validaciones: minimo 10 puntos y en enteros, al menos una apuesta y como mucho una por categoria, podio sin repetir piloto, y solo pilotos inscritos y activos.
- [ ] El total apostado no pasa de los puntos disponibles: los tuyos, o 100 si tienes menos.
- [ ] Solo se juega la carrera abierta: a la hora exacta del cierre ya da 409.
- [ ] Comodin: uno por temporada; no se puede quitar ni borrar la jugada que lo lleva.
- [ ] Si una apuesta no es valida, no se guarda nada de la jugada.
- [ ] GET /api/carreras/:id/jugadas y GET /api/jugadas/:id: la tuya siempre; las de los demas, solo con resultado (si no, 404).
- [ ] GET /api/usuarios/yo/jugadas: tu historial.
- [ ] Logs de jugadas creadas, cambiadas y borradas, y del comodin.

Servidor: grupos

- [ ] POST /api/grupos: crear un grupo, con su codigo de invitacion.
- [ ] POST /api/grupos/unirse: entrar con el codigo o el enlace.

- [ ] GET /api/grupos/:id: el grupo y sus miembros, solo para sus miembros (si no, 404).
- [ ] DELETE /api/grupos/:id/miembros/yo: salir del grupo. El creador no puede salir sin borrarlo.
- [ ] Solo el creador: expulsar (DELETE /api/grupos/:id/miembros/:usuarioId), regenerar el codigo (POST /api/grupos/:id/codigo) y borrar el grupo (DELETE /api/grupos/:id). Cualquier otro: 403.
- [ ] Logs de grupos creados y borrados, uniones y expulsiones.

Pantallas

- [ ] Proxima carrera: cuenta atras, cuantos han jugado y acceso a tu jugada, actualizado con polling.
- [ ] Formulario de jugada: las cinco categorias con piloto y puntos, los puntos disponibles y el total siempre a la vista, y confirmacion antes de activar el comodin.
- [ ] Historial de tus jugadas, dentro de tu perfil.
- [ ] Mis grupos: crear uno, unirse con un codigo y entrar en cada grupo.
- [ ] Pagina de cada grupo: miembros, el codigo para invitar y, para el creador, expulsar, regenerar el codigo y borrar, con confirmacion. La clasificacion del grupo la pone Alex.

Pruebas

- [ ] Crear, ver, cambiar y borrar una jugada antes del cierre, comprobando la base de datos.
- [ ] Un segundo antes del cierre se acepta; a la hora exacta, 409.
- [ ] Jugar una carrera pendiente: 409.
- [ ] Menos de 10 puntos, decimales, ninguna apuesta, dos en la misma categoria, piloto repetido en el podio o piloto inactivo: 400.
- [ ] Con 40 puntos se pueden apostar 100; con 150 no se pueden apostar 160.
- [ ] Segundo comodin, quitar el comodin o borrar su jugada: 409.
- [ ] Cambiar o borrar la jugada de otro: 403. Verla antes del resultado: 404.
- [ ] El admin intenta jugar o entrar en un grupo: 403.
- [ ] Dos peticiones a la vez que juntas pasarian de los puntos disponibles: solo se acepta una.
- [ ] Grupos: crear, unirse con el codigo y salir; un codigo regenerado deja de valer el anterior.
- [ ] Un no miembro pide un grupo: 404. Un miembro que no es el creador intenta expulsar o borrar: 403.

Alex: puntos y clasificaciones

Alex lleva los puntos: el cierre, los cobros, las correcciones, las cancelaciones y las clasificaciones, la general y la de cada grupo. Necesita de Marko las jugadas, la fase de cada carrera y los miembros de cada grupo.

Lo primero, para los demas (semana 2)

- [ ] La tabla de movimientos y la funcion que da los puntos de un usuario en una temporada (la suma de sus movimientos) y sus puntos disponibles. La usan Marko y Abraham.

Servidor

- [ ] Procesar el cierre en la primera peticion que llega despues de la hora: subir a 100 a quien ha jugado y tiene menos, y descontar lo apostado. Una sola vez por carrera, aunque lleguen dos peticiones a la vez.
- [ ] POST /api/carreras/:id/resultado, solo el admin y con la carrera cerrada: guardar el resultado y cobrar x2, x3 el podio exacto y el doble con comodin.
- [ ] PUT /api/carreras/:id/resultado: corregirlo, deshaciendo los cobros anteriores y haciendo los nuevos.
- [ ] La anulacion de una carrera cancelada: devolver lo apostado, deshacer el minimo y los cobros y devolver el comodin. La llama Alvaro.
- [ ] GET /api/carreras/:id/resultado.
- [ ] GET /api/temporadas/:id/clasificacion: por puntos, empates 1, 2, 2, 4 ordenados por nombre, los "sin jugar" al final y sin el admin.
- [ ] GET /api/grupos/:id/clasificacion: la misma, con solo los miembros del grupo y solo para ellos (si no, 404).
- [ ] Logs de cierres, resultados publicados o corregidos y anulaciones.

Pantallas

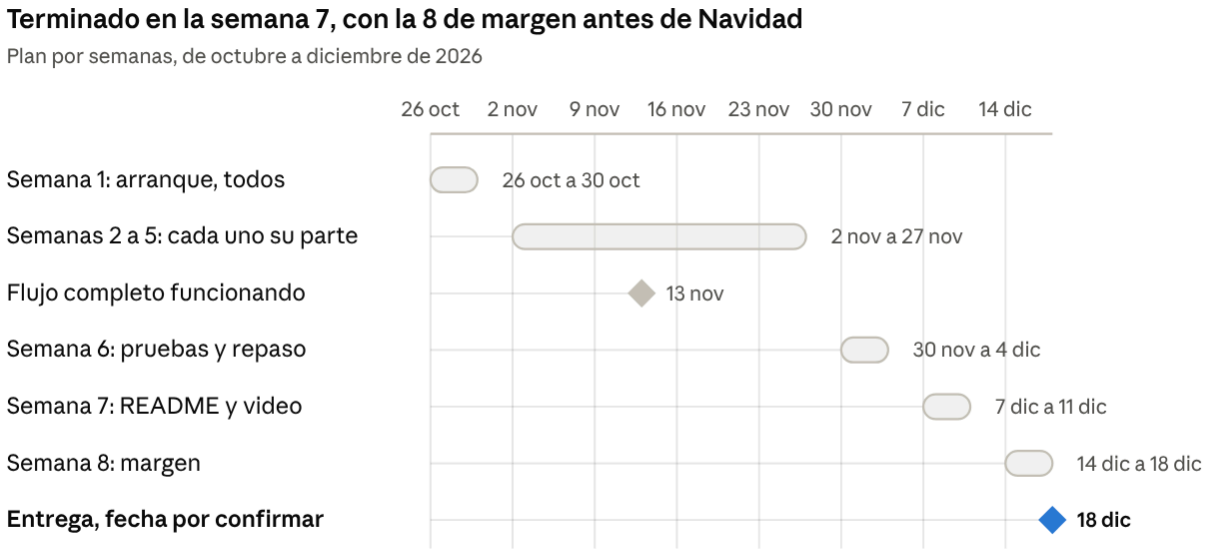
- [ ] Clasificacion general con selector de temporada: tabla en escritorio y tarjetas en movil, actualizada con polling.
- [ ] Clasificacion de cada grupo, dentro de la pagina del grupo.
- [ ] Resultado de una carrera, con las jugadas de todos, sus aciertos y lo que ha cobrado cada uno.
- [ ] Admin: publicar y corregir el resultado, con confirmacion.

Pruebas

- [ ] El minimo de 100 solo se aplica a quien ha jugado, y el cierre no se procesa dos veces.

- [ ] Antes del cierre, apostar no cambia la clasificacion; despues, refleja lo descontado.
- [ ] Cobros: acertada x2, fallada nada, podio exacto x3, comodin x4 y podio con comodin x6.
- [ ] Publicar con la carrera abierta: 409. Publicar dos veces no paga doble.
- [ ] Corregir rehace los cobros, aunque alguien quede en negativo.
- [ ] Cancelar anula todo y devuelve el comodin.
- [ ] Los puntos de cada usuario coinciden con la suma de sus movimientos.
- [ ] Clasificacion general: empates, "sin jugar" y el admin fuera.
- [ ] Clasificacion de un grupo: solo sus miembros, con los mismos puntos que la general; un no miembro recibe 404.
- [ ] Flujo completo: dos usuarios juegan, se cierra, se publica el resultado y la clasificacion queda como debe.

Calendario hasta Navidad



calendario · 8 semanas hasta el 18 de diciembre

En la semana 1 se cierran entre todos el esquema de la BD, los endpoints y el formato de errores, y se deja el repositorio con Docker funcionando. En la semana 3 el flujo completo tiene que funcionar, aunque sea feo.

Pendiente

- La fecha exacta de entrega. El calendario supone unas 8 semanas, hasta el 18 de diciembre.
- El sistema de evaluacion, que segun el PDF se concretara mas adelante.